ARCHITECTURE REVIEW BOARD • TECHNICAL SPECIFICATION

Enterprise Multi-Tenant & Multi-Branch Architecture

Defensible System Design, Data Isolation Guarantees, Schema Contracts, and Execution Blueprint

SYSTEM TARGET	ARCHITECTURE VERSION	EVALUATION STANDARD	STATUS
Field Force Master Platform	**v2.0-MultiTenant**	**Playbook Gate 3 & 5 Criteria**	READY FOR APPROVAL

1. Executive Summary & Objective

The objective of this specification is to transform the standalone Field Service Management (FSM) core platform into a **B2B Enterprise Multi-Tenant SaaS platform** capable of supporting hundreds of independent corporate client organizations. Furthermore, the architecture solves the critical enterprise organizational requirement: **operating multi-branch and regional divisions under a single corporate umbrella without segregating them into disconnected organizations**.

This document provides the technical justification, schema blueprints, security invariants, and runtime execution patterns for engineering review and mentor sign-off prior to codebase modification.

2. Audited Subsystem Inventory & Gap Reconciliation

An audit of the platform codebase identified 53 persistent tables across all operational, commercial, compliance, and analytical modules. The multi-tenant refactoring reconciles structural gaps across all subsystems:

Subsystem	Baseline Entities	Multi-Tenant Isolation & Remedy
Core FSM & Org	`employees`, `office_locations`, `attendance`, `breaks`	Lacked `tenant_id`; singleton config stored in `system_config(id=1)`. Refactored into tenant-scoped tables and dedicated control plane.
Field Travel & Work	`field_sessions`, `field_visits`, `leave_requests`, `leave_balances`, `tasks`	Zero tenant foreign keys. Added non-nullable `tenant_id` and PostgreSQL Row-Level Security (RLS) enforcement on all mutations.
CRM: Leads & Deals	`customers` (unified), `leads`, `lead_history`	Consolidated doctor/hospital directories into canonical `fsm_app.customers`. Deals scoped by `tenant_id` and optional `branch_id`.
CRM: Follow-ups	`followups`, `followup_history`	Preserved Invariant 3 (standalone routine commitments without fake leads). Added `tenant_id` and composite due-date indexes.
Expense Management & Anti-Fraud	`expenses`, `expense_items`, `fuel_mileage_logs`, `disbursements`	Cross-references `field_sessions.accumulated_distance_km` for automated fuel fraud detection under tenant-scoped expense approval chains.
Executive Analytics & KPIs	`executive_kpi_snapshots`, `zonal_strike_rates`, `funnel_conversion_caches`	Pre-aggregated summary cache partitioned by `(tenant_id, branch_id)` enabling sub-second call vs. expense burn analysis.
Monthly Targets	12 tables including `targets`, `assignments`, `progress_entries`, `outbox`	Compound unique keys lacked tenant discriminator; outbox sweeper context-blind. Added `tenant_id` to queue and unique constraints.

Subsystem	Baseline Entities	Multi-Tenant Isolation & Remedy
Revenue Leakage	10 tables including `rules`, `alerts`, `alert_observations`, `actions`	Rule codes and partial unique indexes were global. Converted to `UNIQUE (tenant_id, rule_code)` and tenant-scoped scanner sweeps.
AI Route Engine	`daily_sales_plans`, `items`, `obligations`, `revisions`, `activity_history`	Switched to canonical `customers`; sales plan versions scoped to `(tenant_id, employee_id, date, version)`.

3. Architecture Strategy: Model Comparison & Selection

Three multi-tenancy deployment topologies were evaluated against operational cost, maintenance complexity, and security rigor:

Evaluation Vector	A: Shared DB, Shared Schema (RLS) — SELECTED	B: Schema per Tenant	C: Database per Tenant
Data Isolation Mechanism	Logical via non-nullable `tenant_id` + PostgreSQL RLS policies.	Logical/Physical via PostgreSQL `search_path` per tenant.	Physical network and process isolation per database cluster.
Migration & Schema Drift	**Lowest:** Single Alembic migration step updates all client tables at once.	**High:** Migrations must loop across hundreds of schemas; failures cause state drift.	**Extreme:** Complex orchestration required to patch N independent databases.
Multi-Branch Rollups	**Native:** Cross-branch aggregation and national reports execute via standard SQL.	**Complex:** Requires cross-schema joins or foreign data wrappers.	**Impractical:** Requires distributed ETL pipelines or analytical warehouses.
Operational Cost	**Lowest:** Single connection pool, shared cache, lean hardware footprint.	Medium: PostgreSQL shared catalog bloat; connection pool fragmentation.	High: Idle memory waste, multiple active connection pools and replicas.

> **Architectural Decision Record (ADR-01):** Pattern A (Shared Database with PostgreSQL Row-Level Security) is selected. It provides strict database-level isolation guarantees while allowing native cross-branch rollups and maintainable zero-drift migrations.

4. Organizational Hierarchy: Tenant vs. Branch Scoping

To prevent the critical failure of treating regional offices as separate billing companies, the architecture models **Tenancy** and **Branching** as orthogonal concerns:

- **Tenant (`fsm_system.tenants`):** Represents the corporate paying organization (e.g., *Zydus Healthcare*). It is the boundary of legal contracts, billing, platform security, and RLS partition.
- **Branch (`fsm_app.branches`):** Represents an operational or geographical division within the tenant (e.g., *West Region - Mumbai*, *North Region - Delhi*). It owns local office geofences, local reporting hierarchies, and regional quotas.

Role Authorization & Scoping Matrix

Role	Tenant Scope	Branch Scope	Visibility & Mutation Authority
Executive Leadership / GM	Enforced (`tenant_id`)	**All Branches** (`branch_id IS NULL`)	National performance analytics, cross-branch targets, national leaderboard, global audit logs.
Zonal Sales Manager (ZSM)	Enforced (`tenant_id`)	**Assigned Branch**	Full visibility across regional territories, branch targets, manager escalations, local inventory.
Regional Sales Manager (RSM)	Enforced (`tenant_id`)	**Assigned Territory Cluster**	Downward hierarchy reporting tree (MRs), leave approvals, high-value expense reviews, task dispatch.
Medical Representative (MR)	Enforced (`tenant_id`)	**Assigned Territory Beat**	Personal attendance punch-in, scheduled visits, CRM leads, personal monthly quota progress.

4.1 Multi-Level Account Provisioning & Administrative Lifecycle

The platform enforces a strict separation between **Platform Super-Admin** governance (tenant onboarding, branch setup, root admin provisioning) and **Tenant Admin** workforce operations (employee onboarding, manager reporting, and territory assignment):

Administrative Level	Target Scope & Operations	Workflow Execution & Invariants
Level 1: Platform Super-Admin *(SaaS Platform Operations)*	• `POST /admin/platform/tenants` • `POST /admin/platform/tenants/{id}/branches` • Control Plane (`fsm_system`)	**New Org:** Provisions corporate tenant, creates primary branch (e.g., HQ), and initializes the root Tenant Admin. **Existing Org:** Attaches a new regional branch under the existing `tenant_id`. Never forks an existing client into a disconnected tenant.
Level 2: Tenant Admin *(Client HR / Sales Operations)*	• `POST /api/employees/` • `POST /api/employees/bulk-upload` • Tenant Data Plane (`fsm_app`)	Logs into tenant portal (`X-Tenant-Slug`). Creates management chain (ZSMs, RSMs) and field reps (MRs), assigning each to a specific `branch_id` and `reporting_to_id`.

Cross-Tenant Hierarchy Integrity Trigger

To prevent corrupt reporting trees where an employee reports to a manager from another company, a PostgreSQL trigger enforces tenant integrity at the database layer:

```
CREATE OR REPLACE FUNCTION fsm_app.validate_employee_tenant_integrity() RETURNS TRIGGER AS $$ BEGIN IF NEW.reporting_to_id IS NOT NULL THEN IF NOT EXISTS ( SELECT 1 FROM fsm_app.employees WHERE id = NEW.reporting_to_id AND tenant_id = NEW.tenant_id ) THEN RAISE EXCEPTION 'Integrity Error: Reporting manager must belong to the exact same tenant.'; END IF; END IF; RETURN NEW; END; $$ LANGUAGE plpgsql; CREATE TRIGGER trg_employee_tenant_integrity BEFORE INSERT OR UPDATE ON fsm_app.employees FOR EACH ROW EXECUTE FUNCTION fsm_app.validate_employee_tenant_integrity();
```

5. Persistent Database Contracts & Data Schemas

The database is partitioned into three purposeful schemas to separate administrative platform concerns from transactional multi-tenant operations:

5.1 Control Plane Schema (`fsm_system`)

Houses platform-wide client directory, dynamic feature entitlements, and white-label branding profiles:

```
CREATE SCHEMA IF NOT EXISTS fsm_system; CREATE TABLE fsm_system.tenants ( id UUID PRIMARY KEY DEFAULT gen_random_uuid(), slug VARCHAR(63) NOT NULL UNIQUE, legal_name VARCHAR(255) NOT NULL, trade_name VARCHAR(255), is_active BOOLEAN NOT NULL DEFAULT TRUE, subscription_tier VARCHAR(50) NOT NULL DEFAULT 'standard', created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(), updated_at TIMESTAMPTZ NOT NULL DEFAULT NOW() ); CREATE TABLE fsm_system.tenant_features ( id UUID PRIMARY KEY DEFAULT gen_random_uuid(), tenant_id UUID NOT NULL REFERENCES fsm_system.tenants(id) ON DELETE CASCADE, feature_key VARCHAR(50) NOT NULL, is_enabled BOOLEAN NOT NULL DEFAULT TRUE, CONSTRAINT uq_tenant_feature UNIQUE (tenant_id, feature_key) ); CREATE TABLE fsm_system.tenant_branding ( tenant_id UUID PRIMARY KEY REFERENCES fsm_system.tenants(id) ON DELETE CASCADE, brand_name VARCHAR(100) NOT NULL, logo_url TEXT, primary_color VARCHAR(20) DEFAULT '#1e3a8a', terminology_mapping JSONB NOT NULL DEFAULT '{"target":"Doctor","visit":"DCR Call"}'::jsonb );
```

5.2 Operational Data Plane Schema (`fsm_app`)

All core business entities reside in `fsm_app`. Below are the key foundational entity specifications:

```
       CREATE SCHEMA IF NOT EXISTS fsm_app; -- 1. Operational Branches CREATE TABLE
fsm_app.branches ( id UUID PRIMARY KEY DEFAULT gen_random_uuid(), tenant_id UUID NOT NULL
REFERENCES fsm_system.tenants(id) ON DELETE CASCADE, name VARCHAR(150) NOT NULL, branch_code
VARCHAR(50) NOT NULL, city VARCHAR(100) NOT NULL, is_head_office BOOLEAN NOT NULL DEFAULT
FALSE, is_active BOOLEAN NOT NULL DEFAULT TRUE, created_at TIMESTAMPTZ NOT NULL DEFAULT
NOW(), CONSTRAINT uq_branches_tenant_code UNIQUE (tenant_id, branch_code) ); -- 2. Employees
& Hierarchy CREATE TABLE fsm_app.employees ( id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
tenant_id UUID NOT NULL REFERENCES fsm_system.tenants(id) ON DELETE CASCADE, branch_id UUID
REFERENCES fsm_app.branches(id) ON DELETE SET NULL, -- NULL = National Scope
is_tenant_root_admin BOOLEAN NOT NULL DEFAULT FALSE, email VARCHAR(255) NOT NULL, full_name
VARCHAR(200) NOT NULL, role VARCHAR(50) NOT NULL, reporting_to_id UUID REFERENCES
fsm_app.employees(id) ON DELETE SET NULL, territory VARCHAR(150), is_active BOOLEAN NOT NULL
DEFAULT TRUE, created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(), CONSTRAINT
uq_employees_tenant_email UNIQUE (tenant_id, email) ); -- 3. Unified Customer Master
(Harmonizing CRM & Route Planning) CREATE TABLE fsm_app.customers ( id UUID PRIMARY KEY
DEFAULT gen_random_uuid(), tenant_id UUID NOT NULL REFERENCES fsm_system.tenants(id) ON
DELETE CASCADE, branch_id UUID REFERENCES fsm_app.branches(id) ON DELETE SET NULL, name
VARCHAR(200) NOT NULL, customer_type VARCHAR(50) NOT NULL DEFAULT 'Doctor', phone
VARCHAR(20), location_city VARCHAR(100), latitude DOUBLE PRECISION, longitude DOUBLE
PRECISION, is_active BOOLEAN NOT NULL DEFAULT TRUE, created_at TIMESTAMPTZ NOT NULL DEFAULT
NOW() ); -- 4. Commercial Leads (Opportunities) CREATE TABLE fsm_app.leads ( id UUID PRIMARY
KEY DEFAULT gen_random_uuid(), tenant_id UUID NOT NULL REFERENCES fsm_system.tenants(id) ON
DELETE CASCADE, branch_id UUID REFERENCES fsm_app.branches(id) ON DELETE SET NULL,
employee_id UUID NOT NULL REFERENCES fsm_app.employees(id) ON DELETE CASCADE, created_by_id
UUID NOT NULL REFERENCES fsm_app.employees(id) ON DELETE CASCADE, customer_id UUID
REFERENCES fsm_app.customers(id) ON DELETE SET NULL, field_visit_id UUID REFERENCES
fsm_app.field_visits(id) ON DELETE SET NULL, title VARCHAR(300) NOT NULL, lead_type
VARCHAR(80), stage VARCHAR(50) NOT NULL DEFAULT 'NEW', priority VARCHAR(20) NOT NULL DEFAULT
'MEDIUM', expected_value DOUBLE PRECISION, actual_converted_value DOUBLE PRECISION,
expected_close_date DATE, customer_response TEXT, conversion_outcome TEXT, loss_reason TEXT,
notes TEXT, is_archived BOOLEAN NOT NULL DEFAULT FALSE, created_at TIMESTAMPTZ NOT NULL
DEFAULT NOW(), updated_at TIMESTAMPTZ NOT NULL DEFAULT NOW() ); CREATE INDEX
ix_leads_tenant_emp_stage ON fsm_app.leads(tenant_id, employee_id, stage); -- 4b. Immutable
Lead Audit History CREATE TABLE fsm_app.lead_history ( id UUID PRIMARY KEY DEFAULT
gen_random_uuid(), tenant_id UUID NOT NULL REFERENCES fsm_system.tenants(id) ON DELETE
CASCADE, lead_id UUID NOT NULL REFERENCES fsm_app.leads(id) ON DELETE CASCADE, actor_id UUID
NOT NULL REFERENCES fsm_app.employees(id), event_type VARCHAR(50) NOT NULL, field_name
VARCHAR(50), old_value TEXT, new_value TEXT, notes TEXT, occurred_at TIMESTAMPTZ NOT NULL
DEFAULT NOW() ); -- 4c. Follow-ups (Action Commitments & Standalone Tasks) CREATE TABLE
fsm_app.followups ( id UUID PRIMARY KEY DEFAULT gen_random_uuid(), tenant_id UUID NOT NULL
REFERENCES fsm_system.tenants(id) ON DELETE CASCADE, branch_id UUID REFERENCES
fsm_app.branches(id) ON DELETE SET NULL, employee_id UUID NOT NULL REFERENCES
fsm_app.employees(id) ON DELETE CASCADE, created_by_id UUID NOT NULL REFERENCES
fsm_app.employees(id) ON DELETE CASCADE, lead_id UUID REFERENCES fsm_app.leads(id) ON DELETE
SET NULL, -- Nullable for standalone tasks! customer_id UUID REFERENCES
fsm_app.customers(id) ON DELETE SET NULL, field_visit_id UUID REFERENCES
fsm_app.field_visits(id) ON DELETE SET NULL, customer_name VARCHAR(200), action_type
VARCHAR(80) NOT NULL DEFAULT 'Call', title VARCHAR(300) NOT NULL, due_at TIMESTAMPTZ NOT
NULL, priority VARCHAR(20) NOT NULL DEFAULT 'MEDIUM', status VARCHAR(30) NOT NULL DEFAULT
'PENDING', outcome TEXT, cancel_reason TEXT, completed_at TIMESTAMPTZ, is_archived BOOLEAN
NOT NULL DEFAULT FALSE, created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(), updated_at
TIMESTAMPTZ NOT NULL DEFAULT NOW(), CONSTRAINT chk_followup_context CHECK (lead_id IS NOT
NULL OR customer_id IS NOT NULL OR field_visit_id IS NOT NULL OR customer_name IS NOT NULL)
); CREATE INDEX ix_followups_tenant_emp_due ON fsm_app.followups(tenant_id, employee_id,
status, due_at); -- 4d. Immutable Follow-up Audit History CREATE TABLE
fsm_app.followup_history ( id UUID PRIMARY KEY DEFAULT gen_random_uuid(), tenant_id UUID NOT
NULL REFERENCES fsm_system.tenants(id) ON DELETE CASCADE, followup_id UUID NOT NULL
REFERENCES fsm_app.followups(id) ON DELETE CASCADE, actor_id UUID NOT NULL REFERENCES
```

Security Guarantee: Application connections lease an unprivileged PostgreSQL role (`fsm_app_user`). If a query is executed without setting `app.current_tenant_id`, PostgreSQL returns an empty set (0 rows) and forbids inserts, eliminating accidental data exposure.

7. Application Runtime Architecture & Context Flow

Every incoming HTTP request traverses an automated context resolution lifecycle before touching business logic:

```
      # Fast-path ASGI Middleware Hook async def dispatch(self, request: Request,
call_next): # 1. Resolve Tenant Slug from Subdomain or Header tenant_slug =
extract_slug(request.headers, request.url.hostname) # 2. Fetch validated UUID from Redis /
fsm_system.tenants tenant_uuid = await resolve_tenant_uuid(tenant_slug)
request.state.tenant_id = tenant_uuid # 3. Lease DB session & Bind PostgreSQL RLS Variable
async with async_session_factory() as session: await session.execute(f"SET LOCAL
app.current_tenant_id = '{tenant_uuid}';") response = await call_next(request) return
response
```

Dynamic Terminology Layer: Rather than physically renaming database tables via `TableRegistry`, the backend returns tenant-specific UI vocabulary (e.g., `Doctor` for pharmaceutical clients vs. `Farmer` for agrochemical clients) loaded from `fsm_system.tenant_branding` into the React front-end Zustand stores.

8. Multi-Tenant Background Worker Protocols

Background worker daemons (Outbox Sweeper, Nightly Target Reconciler, Inactive Revenue Leakage Scanner, and Analytics Aggregator) run outside the FastAPI API process. Isolation is maintained during batch execution:

- **Tenant Iteration Loop:** Daemons query `fsm_system.tenants WHERE is_active = TRUE` to obtain all active tenant scopes.
- **Atomic Session Binding:** Prior to evaluating rules or processing outbox queue items, the worker explicitly runs `SET LOCAL app.current_tenant_id = :tenant_id` for each tenant transaction.
- **Fault Containment:** A processing failure in one tenant's outbox batch triggers a rollback for that tenant only, leaving all other tenant pipelines completely unaffected.

8.1 Multi-Tenant Data Archiving & Retention Strategy

To prevent unbounded transactional database bloat while maintaining strict multi-year statutory compliance across all tenant organizations, data is governed across three storage tiers:

Data Category	Hot Tier (`fsm_app`)	Warm Tier (`fsm_archive`)	Cold Tier & Final Purge
Attendance & Telemetry `attendance`, `field_sessions`	90–180 Days (Active working set)	1–3 Years in partitioned PostgreSQL tables	Export to Parquet / Snappy in S3; delete after 5 years (labor audit window).
Proof Photos & Receipts `punch_in_proof`, `receipt_proof`	30–60 Days	S3 Infrequent Access (6 Mos)	Hard deleted after 365 days; database maintains hash reference only.
Financial Ledgers & Audits `progress_entries`, `audit_log`	Active Quarter + Grace (3 Mos)	7 Years (Statutory Indian tax & corporate ledger requirement)	WORM (Write Once Read Many) immutable cloud archive; pre-aggregated sums kept in `fsm_app`.

Data Category	Hot Tier (`fsm_app`)	Warm Tier (`fsm_archive`)	Cold Tier & Final Purge
CRM Leads & Commitments `leads`, `followups`, `customers`	Active Deals + 12 Mos post-close	5–7 Years in archive schema	Never hard-purged unless tenant requests right-to-be-forgotten / contractual erasure.

9. Architectural Risk Analysis & Trade-Offs

Identified Risk	Severity	Architectural Countermeasure & Verification
Cross-Tenant Data Leakage	CRITICAL	PostgreSQL `FORCE ROW LEVEL SECURITY` is active on all operational tables. Restrictive policies prevent queries from escaping the session variable.
Connection Pool Depletion	MEDIUM	Single shared connection pool with connection leasing. `SET LOCAL` ensures variables are automatically cleared when the connection returns to the pool.
Unique Key Collisions	HIGH	All business unique constraints refactored to compound keys prefixed with `tenant_id` (e.g., `UNIQUE(tenant_id, email)`).
Noisy Neighbor Load	MEDIUM	Compound indexes on `(tenant_id, indexed_column)` guarantee index-only scans partitioned by tenant, avoiding sequential table sweeps.

10. Implementation Roadmap & Mentor Sign-Off Gate

Upon sign-off, implementation will execute in three sequential, test-gated phases:

1. **Phase 1 (Schema & DDL Migration):** Provision `fsm_system`, add `tenant_id` and `branch_id` columns, deploy RLS policies, and migrate customer master into canonical `fsm_app.customers`.
2. **Phase 2 (Gateway & Middleware):** Implement FastAPI `MultiTenantSessionMiddleware`, dynamic database session leasing, and JWT tenant claim validation.
3. **Phase 3 (Daemon & UI Integration):** Refactor CLI workers for tenant-scoped batch runs and inject tenant branding/terminology into React Zustand stores.

Architecture Review Board • Mentor Approval Gate

By signing below, the project mentor and lead architect approve the multi-tenant architecture, data model, two-tier branch hierarchy, and multi-level provisioning workflows as sound, robust, and ready for codebase implementation.

Project Architect / Engineering Lead:
Signature: _____
Date: _____

Senior Mentor / Technical Reviewer:
Signature: _____
Approval Verdict: [ ] APPROVED [ ] REVISIONS NEEDED